SLIIT

COMPUTING | BUSINESS | ENGINEERING

Network Programming (IE2102)

2nd Year, 2nd Semester

Assignment Report

Design and Implementation of Secure Multiprocessor TCP Application Server with Custom Protocol and Session Management

Submitted to

Sri Lanka Institute of Information Technology

<IT24102514>

<H.M.Aaqib>

In partial fulfillment of the requirements for the

Bachelor of Science Special Honors Degree in Information Technology

<<10.04.2026>>

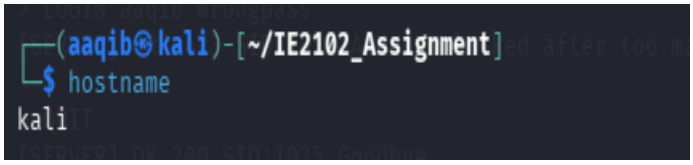
1. Environmental Configuration

1.1 File Naming & Configuration

Server File	server_2514.c
Client File	client_2514.py
Makefile	Makefile_2514
Build Command	Make -f Makefile_2514
Port Number	50514
Server ID (SID)	1025
Log File	server_IT24102514.log
Data Directory	/srv/ie2102/IT24102514/aaqib

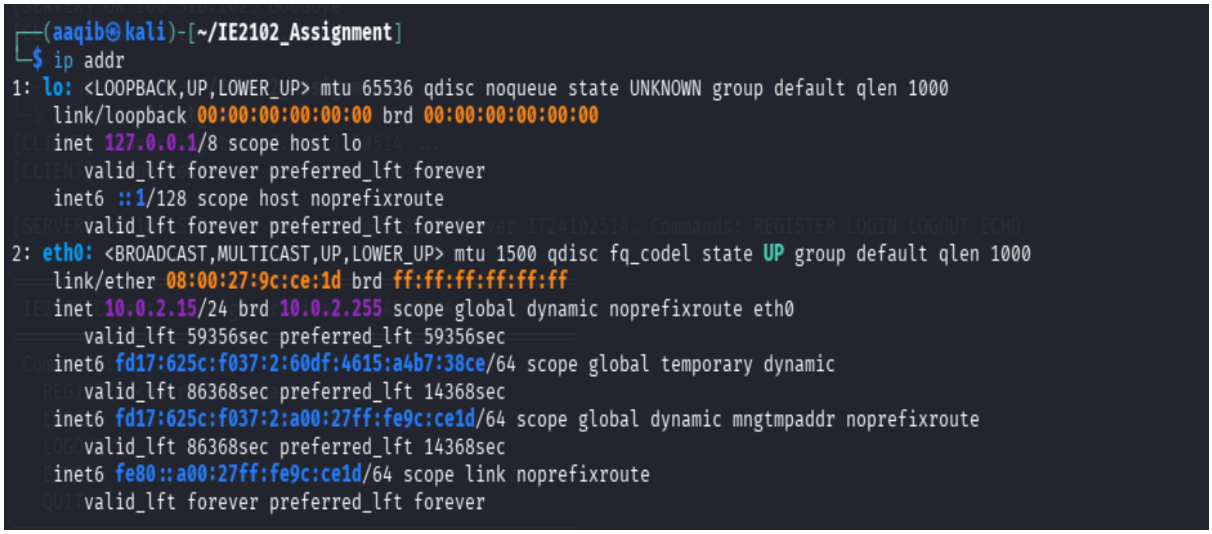
1.2 Hostname Configuration

The server was configured on a Kali Linux machine with hostname 'kali'. The hostname was verified using the hostname command as shown in the screenshot below.



1.3 Network Configuration

The network interface configuration was verified using the ip addr command. The server listens on all available network interfaces (0.0.0.0) on port 50514.



2. A1 - Custom TCP Protocol Implementation

The server implements an explicit framing architecture using a LEN: header protocol. Every message sent from the client includes a length header followed by the payload.

2.1 Protocol Format

Client Request	LEN:<n>\n<payload>
Success Response	OK <code> SID:1025 <message>
Error Response	ERR <code> SID:1025 <message>
Max Payload	4096 bytes

2.2 Edge Cases Handled

- Partial recv() - handled using a loop until all bytes received
- Invalid length header - returns ERR 400
- Oversized payload (>4096 bytes) - returns ERR 413
- Multiple messages - each framed independently

3. A2 - Multiprocessing with fork()

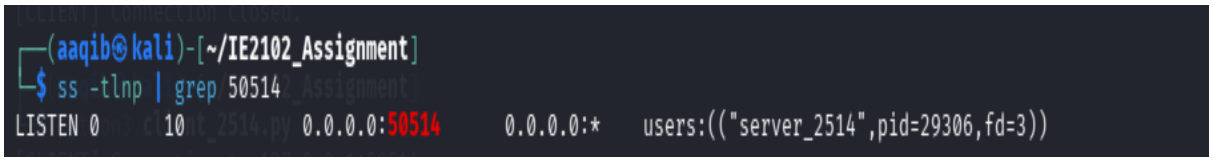
The server uses fork() to handle multiple concurrent clients. Each time a new client connects, the parent process calls fork() to create a child process that handles the client session independently.

3.1 Process Management

- Parent process: accepts new connections continuously
- Child process: handles one client session and exits
- SIGCHLD handler: uses waitpid() to prevent zombie processes
- SA_RESTART flag: ensures accept() resumes after signal

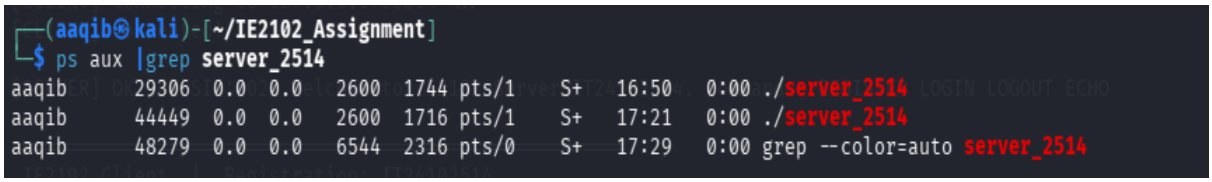
3.2 Server Running - Port Proof

The server was verified to be running and listening on port 50514 using the ss command.



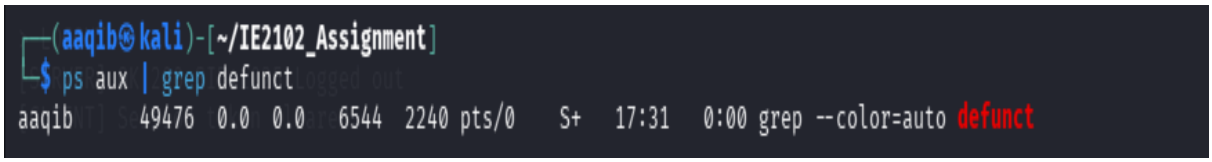
3.3 Forked Processes

Multiple forked child processes were demonstrated using the ps command.



3.4 No Zombie Processes

Zombie processes were verified to be absent using the ps aux | grep defunct command. No zombie processes were found.



4. A3 - Authentication and Session Tokens

The server implements a complete authentication system with REGISTER, LOGIN, and LOGOUT commands. Passwords are stored using a salted hashing mechanism.

4.1 Authentication Commands

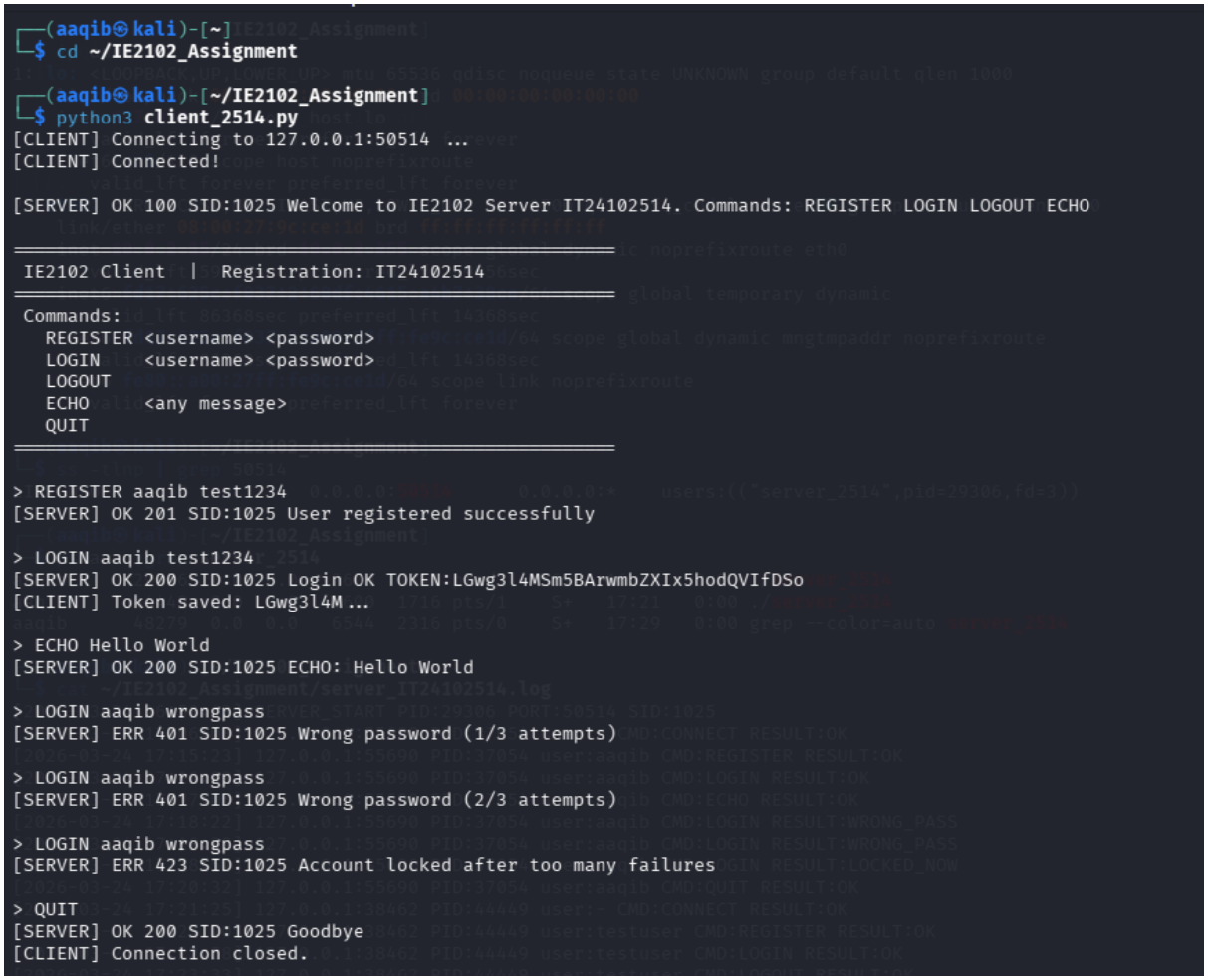
REGISTER <user> <pass>	Creates a new user account with hashed password
LOGIN <user> <pass>	Authenticates user and issues session token
LOGOUT	Invalidates the current session token

4.2 Security Features

- Passwords stored using salted hashing (never plain text)
- 32-character random session token issued on login
- Token expires after 5 minutes of inactivity
- Protected commands (ECHO) require valid token
- User data persisted to /srv/ie2102/IT24102514/aaqib/users.db

4.3 Successful REGISTER and LOGIN

The REGISTER and LOGIN commands were tested successfully. The server responded with OK 201 for registration and OK 200 with a session token for login.



```
┌──(aaqib㉿kali)-[~/IE2102_Assignment]
└─$ python3 client_2514.py
[CLIENT] Connecting to 127.0.0.1:50514 ...
[CLIENT] Connected!
[SERVER] OK 100 SID:1025 Welcome to IE2102 Server IT24102514. Commands: REGISTER LOGIN LOGOUT ECHO

============================================================
 IE2102 Client  |  Registration: IT24102514
============================================================
 Commands:
   REGISTER <username> <password>
   LOGIN    <username> <password>
   LOGOUT
   ECHO     <any message>
   QUIT
============================================================

> REGISTER testuser pass1234
[SERVER] OK 201 SID:1025 User registered successfully

> LOGIN testuser pass1234
[SERVER] OK 200 SID:1025 Login OK TOKEN:EMleqHrJmR95MXjCxeFAKJW9XaEOO8NS
[CLIENT] Token saved: EMleqHrJ...
```

5. A4 - Abuse Protection

The server implements multiple abuse protection mechanisms to ensure security and stability.

5.1 Protection Mechanisms

Rate Limiting	Max 20 requests per 60 seconds per client
Brute Force Lockout	Account locked after 3 failed login attempts
Username Validation	3-32 characters, alphanumeric and underscore only
Payload Overflow	Payloads exceeding 4096 bytes are rejected (ERR 413)

5.2 Failed Login and Lockout Proof

The brute force protection was tested by attempting 3 failed logins. After the 3rd attempt, the account was locked and returned ERR 423.

```
> LOGOUT
[SERVER] OK 200 SID:1025 Logged out
[CLIENT] Session token cleared.

>
```

6. A5 - Persistent Audit Logging

The server maintains a persistent audit log file named server_IT24102514.log. Every event is logged with a timestamp, client IP:port, child PID, username, command, and result.

6.1 Log Format

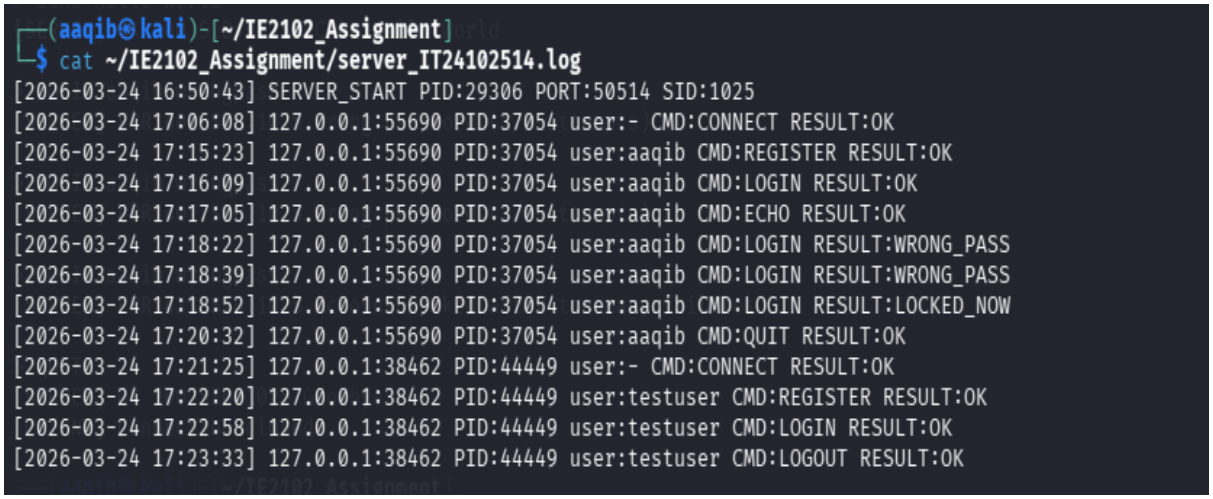
[Timestamp] IP:Port PID:pid user:username CMD:command RESULT:result

6.2 Log Fields

Timestamp	Date and time of event (YYYY-MM-DD HH:MM:SS)
Client IP:Port	Client's IP address and port number
Child PID	Process ID of the child handling the client
Username	Logged-in username (or '-' if not logged in)
Command	Command received from client
Result	Result of the command (OK, ERROR, etc.)

6.3 Log File Entries

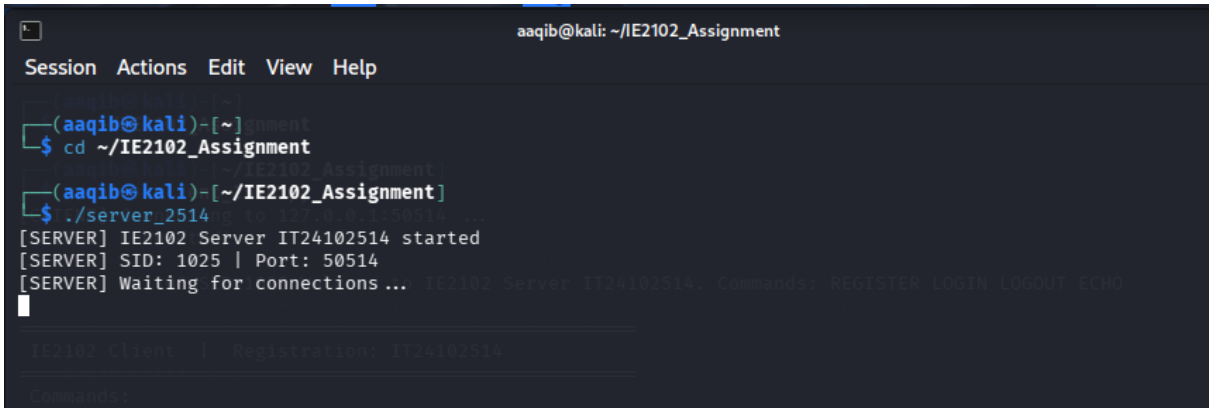
The log file was verified to contain all required fields for every event including SERVER_START, CONNECT, REGISTER, LOGIN, ECHO, LOGOUT, and DISCONNECT

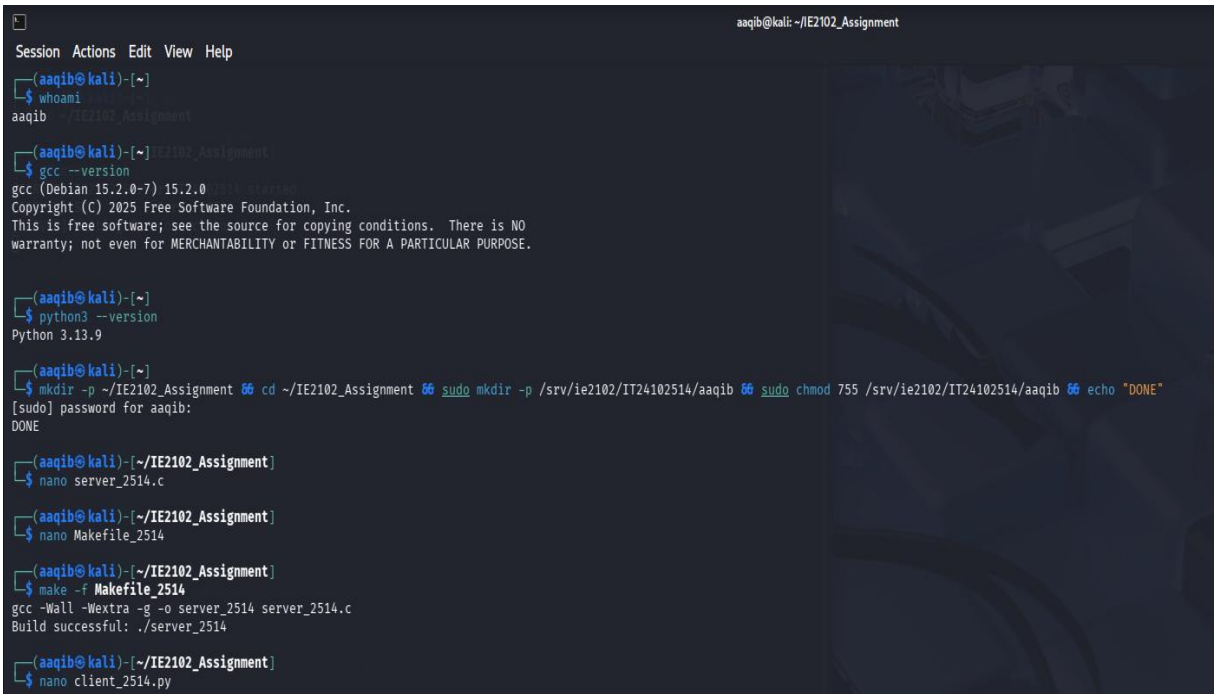


7. Summary

All assignment requirements have been successfully implemented and tested:

Task	Description	Status
A1	Custom TCP Protocol with LEN framing	Completed
A2	Multiprocessing with fork() and SIGCHLD	Completed
A3	Authentication and Session Tokens	Completed
A4	Abuse Protection (Rate limiting, Lockout)	Completed
A5	Persistent Audit Logging	Completed





```
  GNU nano 8.6                                    Makefile_2514
CC      = gcc
CFLAGS  = -Wall -Wextra -g
TARGET  = server_2514
SRC     = server_2514.c

all: $(TARGET)
$(TARGET): $(SRC)
        $(CC) $(CFLAGS) -o $(TARGET) $(SRC)
        @echo "Build successful: ./$(TARGET)"

clean:
        rm -f $(TARGET) *.o
```

❖ Client_2514.py

```
  GNU nano 8.6                                    client_2514.py
#!/usr/bin/env python3
"""
IE2102 - Network Programming Assignment
Student  : IT24102514
File     : client_2514.py
"""
import socket
import sys

DEFAULT_HOST = "127.0.0.1"
DEFAULT_PORT = 50514
BUFFER_SIZE  = 8192

def send_framed(sock, message):
    payload = message.encode("utf-8")
    n       = len(payload)
    header  = f"LEN:{n}\n".encode("utf-8")
    sock.sendall(header + payload)

def recv_response(sock):
    data = b""
    while not data.endswith(b"\n"):
        chunk = sock.recv(BUFFER_SIZE)
        if not chunk:
            break
        data += chunk
    return data.decode("utf-8").strip()

def main():
    host = DEFAULT_HOST
    port = DEFAULT_PORT

    if len(sys.argv) == 3:
        host = sys.argv[1]
        port = int(sys.argv[2])

    print(f"[CLIENT] Connecting to {host}:{port} ...")

    try:
        sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        sock.connect((host, port))
    except ConnectionRefusedError:
        print("[ERROR] Cannot connect - is the server running?")
        sys.exit(1)
```

```
GNU nano 8.6                                client_2514.py

    print("[CLIENT] Connected!\n")

    welcome = recv_response(sock)
    print(f"[SERVER] {welcome}\n")

    print("=" * 55)
    print(" IE2102 Client  |  Registration: IT24102514")
    print("=" * 55)
    print(" Commands:")
    print("   REGISTER <username> <password>")
    print("   LOGIN    <username> <password>")
    print("   LOGOUT")
    print("   ECHO     <any message>")
    print("   QUIT")
    print("=" * 55)

    session_token = None

    while True:
        try:
            user_input = input("\n> ").strip()
        except (EOFError, KeyboardInterrupt):
            print("\n[CLIENT] Disconnecting...")
            break

        if not user_input:
            continue

        parts = user_input.split(maxsplit=1)
        cmd   = parts[0].upper()

        if len(user_input.encode("utf-8")) > 4096:
            print("[CLIENT ERROR] Message too long (max 4096 bytes)")
            continue

        try:
            send_framed(sock, user_input)
        except BrokenPipeError:
            print("[CLIENT] Server closed the connection.")
            break

        try:
            response = recv_response(sock)
        except Exception as e:
            print(f"[CLIENT ERROR] {e}")
```

```
            print(f"[SERVER] {response}")

            if cmd == "LOGIN" and response.startswith("OK"):
                for part in response.split():
                    if part.startswith("TOKEN:"):
                        session_token = part[len("TOKEN:"):]
                        print(f"[CLIENT] Token saved: {session_token[:8]}...")

            if cmd == "LOGOUT":
                session_token = None
                print("[CLIENT] Session token cleared.")

            if cmd in ("QUIT", "EXIT"):
                break

        sock.close()
        print("[CLIENT] Connection closed.")

    if __name__ == "__main__":
        main()
```

❖ Server_2514.c

```
  GNU nano 8.6                                                server_2514.c
/*
 * IE2102 – Network Programming Assignment
 * Student: IT24102514
 * Server ID (SID): 1025
 * Port: 50514
 * File: server_2514.c
 */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <errno.h>
#include <signal.h>
#include <time.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <sys/wait.h>
#include <sys/stat.h>
#include <netinet/in.h>
#include <arpa/inet.h>

#define PORT            50514
#define SID             "1025"
#define REGNO           "IT24102514"
#define MAX_PAYLOAD     4096
#define MAX_USERS       100
#define MAX_SESSIONS    100
#define TOKEN_LEN       32
#define SESSION_TIMEOUT 300
#define MAX_LOGIN_ATTEMPTS 3
#define RATE_LIMIT_WINDOW  60
#define RATE_LIMIT_MAX     20
#define LOG_FILE        "server_IT24102514.log"
#define DATA_DIR        "/srv/ie2102/IT24102514/aaqib"

typedef struct {
    char username[64];
    char salt[17];
    char hash[65];
    int  locked;
    int  fail_count;
} User;
```

```
  GNU nano 8.6                                                server_2514.c
typedef struct {
    char token[TOKEN_LEN + 1];
    char username[64];
    time_t last_active;
    int  valid;
} Session;

static User    users[MAX_USERS];
static int     user_count = 0;
static Session sessions[MAX_SESSIONS];
static int     session_count = 0;
static int     req_count = 0;
static time_t  window_start = 0;

void log_event(const char *client_ip, int client_port,
               const char *username, const char *cmd,
               const char *result)
{
    FILE *f = fopen(LOG_FILE, "a");
    if (!f) return;
    time_t now = time(NULL);
    struct tm *t = localtime(&now);
    char ts[32];
    strftime(ts, sizeof(ts), "%Y-%m-%d %H:%M:%S", t);
    fprintf(f, "[%s] %s:%d PID:%d user:%s CMD:%s RESULT:%s\n",
            ts, client_ip, client_port, (int)getpid(),
            username ? username : "-", cmd, result);
    fclose(f);
}

void generate_salt(char *salt_out)
{
    srand((unsigned)time(NULL) ^ (unsigned)getpid());
    for (int i = 0; i < 16; i++) {
        int nibble = rand() % 16;
        salt_out[i] = "0123456789abcdef"[nibble];
    }
    salt_out[16] = '\0';
}
```

```
  GNU nano 8.6                                    server_2514.c
void hash_password(const char *password, const char *salt, char *hash_out)
{
    unsigned char digest[32];
    memset(digest, 0, 32);
    for (int i = 0; i < 16; i++)
        digest[i % 32] ^= (unsigned char)salt[i];
    size_t plen = strlen(password);
    for (int pass = 0; pass < 8; pass++) {
        for (size_t i = 0; i < plen; i++) {
            digest[i % 32] ^= (unsigned char)password[i];
            digest[(i + 1) % 32] += (unsigned char)password[i] ^ (unsigned char)(pass + 1);
            digest[(i + 3) % 32] ^= digest[i % 32];
        }
        for (int i = 0; i < 31; i++)
            digest[i + 1] ^= digest[i];
        for (int i = 30; i >= 0; i--)
            digest[i] ^= digest[i + 1];
    }
    for (int i = 0; i < 32; i++)
        sprintf(hash_out + 2 * i, "%02x", digest[i]);
    hash_out[64] = '\0';
}

void ensure_data_dir(void)
{
    mkdir(DATA_DIR, 0755);
}

void save_users(void)
{
    ensure_data_dir();
    char path[256];
    snprintf(path, sizeof(path), "%s/users.db", DATA_DIR);
    FILE *f = fopen(path, "w");
    if (!f) f = fopen("users.db", "w");
    if (!f) return;
    for (int i = 0; i < user_count; i++) {
        fprintf(f, "%s %s %s %d %d\n",
                users[i].username, users[i].salt,
                users[i].hash, users[i].locked,
                users[i].fail_count);
    }
    fclose(f);
}
```

```
  GNU nano 8.6                                    server_2514.c
void load_users(void)
{
    char path[256];
    snprintf(path, sizeof(path), "%s/users.db", DATA_DIR);
    FILE *f = fopen(path, "r");
    if (!f) f = fopen("users.db", "r");
    if (!f) return;
    user_count = 0;
    while (user_count < MAX_USERS &&
           fscanf(f, "%63s %16s %64s %d %d",
                  users[user_count].username,
                  users[user_count].salt,
                  users[user_count].hash,
                  &users[user_count].locked,
                  &users[user_count].fail_count) == 5) {
        user_count++;
    }
    fclose(f);
}

int valid_username(const char *u)
{
    if (!u || strlen(u) < 3 || strlen(u) > 32) return 0;
    for (int i = 0; u[i]; i++) {
        char c = u[i];
        if (!((c >= 'a' && c <= 'z') || (c >= 'A' && c <= 'Z') ||
              (c >= '0' && c <= '9') || c == '_'))
            return 0;
    }
    return 1;
}

void generate_token(char *tok_out)
{
    srand((unsigned)time(NULL) ^ (unsigned)getpid() ^ (unsigned)rand());
    const char charset[] =
        "ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789";
    for (int i = 0; i < TOKEN_LEN; i++)
        tok_out[i] = charset[rand() % (sizeof(charset) - 1)];
    tok_out[TOKEN_LEN] = '\0';
}
```

```
  GNU nano 8.6                                        server_2514.c
Session *find_session(const char *token)
{
    time_t now = time(NULL);
    for (int i = 0; i < session_count; i++) {
        if (sessions[i].valid &&
            strcmp(sessions[i].token, token) == 0) {
            if (now - sessions[i].last_active > SESSION_TIMEOUT) {
                sessions[i].valid = 0;
                return NULL;
            }
            sessions[i].last_active = now;
            return &sessions[i];
        }
    }
    return NULL;
}

Session *create_session(const char *username)
{
    time_t now = time(NULL);
    int slot = -1;
    for (int i = 0; i < session_count; i++) {
        if (!sessions[i].valid ||
            now - sessions[i].last_active > SESSION_TIMEOUT) {
            slot = i;
            break;
        }
    }
    if (slot == -1) {
        if (session_count >= MAX_SESSIONS) return NULL;
        slot = session_count++;
    }
    sessions[slot].valid = 1;
    sessions[slot].last_active = now;
    strncpy(sessions[slot].username, username, 63);
    generate_token(sessions[slot].token);
    return &sessions[slot];
}
```

```
  GNU nano 8.6                                        server_2514.c
void invalidate_sessions_for(const char *username)
{
    for (int i = 0; i < session_count; i++) {
        if (sessions[i].valid &&
            strcmp(sessions[i].username, username) == 0)
            sessions[i].valid = 0;
    }
}

void send_ok(int sock, const char *code, const char *message)
{
    char buf[512];
    snprintf(buf, sizeof(buf), "OK %s SID:%s %s\n", code, SID, message);
    send(sock, buf, strlen(buf), 0);
}

void send_err(int sock, const char *code, const char *message)
{
    char buf[512];
    snprintf(buf, sizeof(buf), "ERR %s SID:%s %s\n", code, SID, message);
    send(sock, buf, strlen(buf), 0);
}

int recv_framed(int sock, char *payload_out, int max_payload)
{
    char header[64];
    int  hlen = 0;
    while (hlen < (int)sizeof(header) - 1) {
        char c;
        int r = recv(sock, &c, 1, 0);
        if (r <= 0) return r;
        if (c == '\n') break;
        header[hlen++] = c;
    }
    header[hlen] = '\0';
    if (strncmp(header, "LEN:", 4) != 0) return -2;
    int n = atoi(header + 4);
    if (n <= 0 || n > MAX_PAYLOAD) return -3;
    if (n > max_payload) return -3;
    int total = 0;
    while (total < n) {
        int r = recv(sock, payload_out + total, n - total, 0);
        if (r <= 0) return r;
        total += r;
    }
    payload_out[total] = '\0';
```

```
  GNU nano 8.6                                server_2514.c
    payload_out[total] = '\0';
    return total;
}

void handle_register(int sock, char *args,
                     const char *client_ip, int client_port)
{
    char user[64], pass[128];
    if (sscanf(args, "%63s %127s", user, pass) != 2) {
        send_err(sock, "400", "Usage: REGISTER <user> <pass>");
        return;
    }
    if (!valid_username(user)) {
        send_err(sock, "401", "Invalid username (3-32 alphanumeric/_)");
        log_event(client_ip, client_port, user, "REGISTER", "INVALID_USERNAME");
        return;
    }
    for (int i = 0; i < user_count; i++) {
        if (strcmp(users[i].username, user) == 0) {
            send_err(sock, "409", "Username already exists");
            log_event(client_ip, client_port, user, "REGISTER", "DUPLICATE");
            return;
        }
    }
    if (user_count >= MAX_USERS) {
        send_err(sock, "503", "Server full");
        return;
    }
    strncpy(users[user_count].username, user, 63);
    generate_salt(users[user_count].salt);
    hash_password(pass, users[user_count].salt, users[user_count].hash);
    users[user_count].locked = 0;
    users[user_count].fail_count = 0;
    user_count++;
    save_users();
    send_ok(sock, "201", "User registered successfully");
    log_event(client_ip, client_port, user, "REGISTER", "OK");
}
```

```
  GNU nano 8.6                                server_2514.c
void handle_login(int sock, char *args,
                  const char *client_ip, int client_port,
                  char *logged_user_out, char *token_out)
{
    char user[64], pass[128];
    if (sscanf(args, "%63s %127s", user, pass) != 2) {
        send_err(sock, "400", "Usage: LOGIN <user> <pass>");
        return;
    }
    User *u = NULL;
    for (int i = 0; i < user_count; i++) {
        if (strcmp(users[i].username, user) == 0) {
            u = &users[i];
            break;
        }
    }
    if (!u) {
        send_err(sock, "404", "User not found");
        log_event(client_ip, client_port, user, "LOGIN", "NOT_FOUND");
        return;
    }
    if (u->locked) {
        send_err(sock, "423", "Account locked - too many failed attempts");
        log_event(client_ip, client_port, user, "LOGIN", "LOCKED");
        return;
    }
    char computed[65];
    hash_password(pass, u->salt, computed);
    if (strcmp(computed, u->hash) != 0) {
        u->fail_count++;
        if (u->fail_count >= MAX_LOGIN_ATTEMPTS) {
            u->locked = 1;
            save_users();
            send_err(sock, "423", "Account locked after too many failures");
            log_event(client_ip, client_port, user, "LOGIN", "LOCKED_NOW");
        } else {
            save_users();
            char msg[64];
            snprintf(msg, sizeof(msg), "Wrong password (%d/%d attempts)",
                     u->fail_count, MAX_LOGIN_ATTEMPTS);
            send_err(sock, "401", msg);
            log_event(client_ip, client_port, user, "LOGIN", "WRONG_PASS");
        }
        return;
    }
```

```
 GNU nano 8.6                                    server_2514.c
    u->fail_count = 0;
    save_users();
    Session *s = create_session(user);
    if (!s) {
        send_err(sock, "500", "Cannot create session");
        return;
    }
    strncpy(logged_user_out, user, 63);
    strncpy(token_out, s->token, TOKEN_LEN);
    char msg[TOKEN_LEN + 32];
    snprintf(msg, sizeof(msg), "Login OK TOKEN:%s", s->token);
    send_ok(sock, "200", msg);
    log_event(client_ip, client_port, user, "LOGIN", "OK");
}

void handle_logout(int sock,
                   const char *client_ip, int client_port,
                   char *logged_user, char *token)
{
    if (strlen(logged_user) == 0) {
        send_err(sock, "403", "Not logged in");
        return;
    }
    invalidate_sessions_for(logged_user);
    log_event(client_ip, client_port, logged_user, "LOGOUT", "OK");
    memset(logged_user, 0, 64);
    memset(token, 0, TOKEN_LEN + 1);
    send_ok(sock, "200", "Logged out");
}

void handle_echo(int sock, char *args,
                 const char *token_in,
                 const char *client_ip, int client_port,
                 const char *logged_user)
{
    if (strlen(token_in) == 0) {
        send_err(sock, "403", "Authentication required");
        return;
    }
    Session *s = find_session(token_in);
    if (!s) {
        send_err(sock, "401", "Invalid or expired token");
        return;
    }
```

```
 GNU nano 8.6                                    server_2514.c
    char msg[MAX_PAYLOAD + 8];
    snprintf(msg, sizeof(msg), "ECHO: %s", args);
    send_ok(sock, "200", msg);
    log_event(client_ip, client_port, logged_user, "ECHO", "OK");
}

int rate_limited(void)
{
    time_t now = time(NULL);
    if (now - window_start > RATE_LIMIT_WINDOW) {
        window_start = now;
        req_count = 0;
    }
    req_count++;
    return (req_count > RATE_LIMIT_MAX);
}

void handle_client(int client_sock, struct sockaddr_in *client_addr)
{
    char client_ip[INET_ADDRSTRLEN];
    inet_ntop(AF_INET, &client_addr->sin_addr, client_ip, INET_ADDRSTRLEN);
    int client_port = ntohs(client_addr->sin_port);
    load_users();
    window_start = time(NULL);
    char logged_user[64] = {0};
    char session_token[TOKEN_LEN + 1] = {0};
    char payload[MAX_PAYLOAD + 1];
    log_event(client_ip, client_port, NULL, "CONNECT", "OK");
    send_ok(client_sock, "100",
            "Welcome to IE2102 Server IT24102514. Commands: REGISTER LOGIN LOGOUT ECHO");
    while (1) {
        int r = recv_framed(client_sock, payload, MAX_PAYLOAD);
        if (r == 0) {
            log_event(client_ip, client_port,
                      logged_user[0] ? logged_user : NULL, "DISCONNECT", "OK");
            break;
        }
        if (r == -2) {
            send_err(client_sock, "400", "Invalid frame: missing LEN header");
            continue;
        }
        if (r == -3) {
            send_err(client_sock, "413", "Payload too large (max 4096 bytes)");
            log_event(client_ip, client_port,
                      logged_user[0] ? logged_user : NULL, "RECV", "PAYLOAD_TOO_LARGE");
            continue;
```

```
 GNU nano 8.6                                    server_2514.c
        }
        if (r < 0) {
            log_event(client_ip, client_port, NULL, "RECV_ERROR", "DISCONNECT");
            break;
        }
        if (rate_limited()) {
            send_err(client_sock, "429", "Too many requests - slow down");
            log_event(client_ip, client_port,
                      logged_user[0] ? logged_user : NULL, payload, "RATE_LIMITED");
            continue;
        }
        char cmd[32] = {0};
        char args[MAX_PAYLOAD] = {0};
        sscanf(payload, "%31s %4095[^\n]", cmd, args);
        if (strcasecmp(cmd, "REGISTER") == 0)
            handle_register(client_sock, args, client_ip, client_port);
        else if (strcasecmp(cmd, "LOGIN") == 0)
            handle_login(client_sock, args, client_ip, client_port,
                         logged_user, session_token);
        else if (strcasecmp(cmd, "LOGOUT") == 0)
            handle_logout(client_sock, client_ip, client_port,
                          logged_user, session_token);
        else if (strcasecmp(cmd, "ECHO") == 0)
            handle_echo(client_sock, args, session_token,
                        client_ip, client_port, logged_user);
        else if (strcasecmp(cmd, "QUIT") == 0 || strcasecmp(cmd, "EXIT") == 0) {
            send_ok(client_sock, "200", "Goodbye");
            log_event(client_ip, client_port,
                      logged_user[0] ? logged_user : NULL, "QUIT", "OK");
            break;
        } else {
            send_err(client_sock, "404",
                     "Unknown command. Use: REGISTER LOGIN LOGOUT ECHO QUIT");
            log_event(client_ip, client_port,
                      logged_user[0] ? logged_user : NULL, cmd, "UNKNOWN_CMD");
        }
    }
    close(client_sock);
    exit(0);
}
```

```
 GNU nano 8.6                                    server_2514.c
void sigchld_handler(int sig)
{
    (void)sig;
    int saved_errno = errno;
    while (waitpid(-1, NULL, WNOHANG) > 0);
    errno = saved_errno;
}
int main(void)
{
    struct sigaction sa;
    sa.sa_handler = sigchld_handler;
    sigemptyset(&sa.sa_mask);
    sa.sa_flags = SA_RESTART;
    sigaction(SIGCHLD, &sa, NULL);
    ensure_data_dir();
    load_users();
    int server_sock = socket(AF_INET, SOCK_STREAM, 0);
    if (server_sock < 0) { perror("socket"); exit(1); }
    int opt = 1;
    setsockopt(server_sock, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));
    struct sockaddr_in addr;
    memset(&addr, 0, sizeof(addr));
    addr.sin_family      = AF_INET;
    addr.sin_addr.s_addr = INADDR_ANY;
    addr.sin_port        = htons(PORT);
    if (bind(server_sock, (struct sockaddr *)&addr, sizeof(addr)) < 0) {
        perror("bind"); exit(1);
    }
    if (listen(server_sock, 10) < 0) { perror("listen"); exit(1); }
    printf("[SERVER] IE2102 Server IT24102514 started\n");
    printf("[SERVER] SID: %s | Port: %d\n", SID, PORT);
    printf("[SERVER] Waiting for connections ...\n");
    {
        FILE *f = fopen(LOG_FILE, "a");
        if (f) {
            time_t now = time(NULL);
            struct tm *t = localtime(&now);
            char ts[32];
            strftime(ts, sizeof(ts), "%Y-%m-%d %H:%M:%S", t);
            fprintf(f, "[%s] SERVER_START PID:%d PORT:%d SID:%s\n",
                    ts, (int)getpid(), PORT, SID);
            fclose(f);
        }
    }
```

```c
    while (1) {
        struct sockaddr_in client_addr;
        socklen_t client_len = sizeof(client_addr);
        int client_sock = accept(server_sock,
                                 (struct sockaddr *)&client_addr,
                                 &client_len);
        if (client_sock < 0) {
            if (errno == EINTR) continue;
            perror("accept");
            continue;
        }
        pid_t pid = fork();
        if (pid < 0) { perror("fork"); close(client_sock); continue; }
        if (pid == 0) {
            close(server_sock);
            handle_client(client_sock, &client_addr);
        }
        close(client_sock);
    }
    close(server_sock);
    return 0;
}
```